

I know there is a google platform that can make fuzzer compete each other on benchmarks, or so. So you can send your fuzzer to this platform and they will run and assess it against the rest of the fuzzers they have

Thought for 19s >

You're thinking of **FuzzBench** (made by Google). It's basically "fuzzer benchmarking as a service": you add your fuzzer to their platform, they run standardized experiments at scale, and you get a report comparing it to other fuzzers. google.github.io +2

What the platform is (and isn't)

FuzzBench (the "fuzzers compete" one)

- Purpose: **compare fuzzers against each other** on the same set of real-world benchmarks, using consistent methodology. [GitHub](#) +2
- How it works (official flow): you **integrate** your fuzzer into the FuzzBench repo → it gets merged → their service runs experiments → a **published report** compares performance against existing fuzzers (per-benchmark + aggregate). google.github.io
- Benchmarks: there's a documented list of included targets; many are derived from OSS-Fuzz fuzz targets. google.github.io +1

OSS-Fuzz / ClusterFuzz (related, but different)

- **OSS-Fuzz** is continuous fuzzing for open-source projects (find bugs in projects over time), not primarily a "fuzzer tournament." solar.cs.ucl.ac.uk +1

- FuzzBench benchmarks are often based on OSS-Fuzz targets, but the goals differ: **FuzzBench = evaluation**, **OSS-Fuzz = ongoing bug-finding for projects**. solar.cs.ucl.ac.uk

About “binutils apps”



- FuzzBench’s standard benchmark set is documented; **binutils isn’t necessarily a default benchmark** in the main set, but the ecosystem absolutely evaluates on binutils-family tools (e.g., papers/bench suites integrating binutils utilities like `objdump`). google.github.io +2
- Practically: if you care specifically about binutils, you can still use the **FuzzBench tooling/infrastructure** to integrate those targets (either privately in your fork/local runs, or as a contributed benchmark if accepted). GitHub +2

How you’d “send your fuzzer” to it (high-level steps)

1. **Make your fuzzer runnable in their harness model** (usually Docker-based, consistent entrypoints, etc.). google.github.io +1
2. **Add an integration** for your fuzzer to the FuzzBench repository (their docs cover “Adding a fuzzer”). google.github.io +1
3. **Open a Pull Request** to the FuzzBench repo.
4. Once merged, the service **runs experiments and publishes a comparison report** against other fuzzers. google.github.io

Pro tips so your results are taken seriously

- **Be explicit about your “fairness” assumptions:** CPU cores, memory, dictionary usage, seed corpora, and whether you rely on special instrumentation.
- **Expect variance:** fuzzing results can be noisy; FuzzBench is designed to make comparisons more reproducible, but you should still interpret small differences carefully. Google Research +1

- If your goal is “best on binutils,” consider:
 - Running **local FuzzBench experiments** with a binutils benchmark you add yourself.
 - Comparing on both **coverage growth** and **bug/CRASH discovery** depending on what you’re optimizing for.

If you tell me (1) what base fuzzer you built on (AFL++, libFuzzer, honggfuzz, custom) and (2) what’s novel about yours (mutator, scheduler, power schedule, grammar, concolic, etc.), I can suggest the cleanest way to package it for FuzzBench-style evaluation and what metrics to emphasize.

</> Plain text



```
URLs (for your reference)
https://google.github.io/fuzzbench/
https://github.com/google/fuzzbench
https://google.github.io/fuzzbench/reference/benchmarks/
https://research.google/pubs/fuzzbench-an-open-fuzzer-benchmarking-platform-an
https://opensource.googleblog.com/2020/03/fuzzbench-fuzzer-benchmarking-as-ser
https://bughunters.google.com/about/rules/open-source/oss-fuzz-reward-program-
```



Sources